BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL ENGINEERING AND INFORMATION TECHNOLOGY

DOCUMENTATION FOR THE CINEMA DATABASE

AUTHORS: Daniel Prachař, Dalibor Rada
DATE OF CREATION: 20. 10. 2022

Abstract:

Database description

Our database, originally called CinemaVillage, deals with the problem of cinemas.
In our project, we mainly deal with the attributes of cinemas and their employees.
To get a closer look at our database, the Cinema entity is mainly about relationships with entities such as opening hours, addresses of given cinemas, which movies are playing in which cinema.
With the Employee entity, we then solve the problem of which employee works in which cinema, what address the given employee has, or what function he represents.
By the way, as you may notice, we used a similar name for cinemas as company called CinemaCity to fill the database with data, this similarity serves no other purposes than educational.

Table descriptions

cinema:

The cinema table is used to record the branches of our cinemas. We record the name of the cinema, contact email for each branch and finally if there is free parking in front of the cinema or barrier-free access.

- id_cinema = unique identification of the cinema branch
    - DT = serial: Serial is suitable type (and typically used) for a numeric unique identifier
- cinemas_name = the full name of our branch
    - DT = varchar(60): For entering a text string, there is no other option (could be used char, but varchar is more flexible)
- Email = each branch has its own unique email where customers can contact it
    - DT = varchar(45): For entering a text string
- free_parking = describes if the branch has a marked parking lot nearby, which is free for cinema customers,
    - DT in PostgreSQL = Boolean: We used boolean because it was in the assignment. Boolean returns the value true=there is free parking, false=there is no free parking, null = there is no record. However, in our opinion, it would be better to use the enum data type.

    - DT in MySQL = enum: We used enum type because it seems more user-friendly than Boolean. Enum with selection of the values: yes = for free parking and no = for parking that is not free, null = there is no record
- wheelchair_access = this attribute indicates whether you can enter our cinema as a wheelchair user
    - DT in PostreSQL = Boolean: As in the previous case, we only needed the answer true = has wheelchair access and false= has not wheelchair access
    - DT in MySQL = enum: even here we found the use of the enum type more appropriate

address:

The address table is used to store the following attributes: id, city name, street, house number and zip-code. These attributes are linked to our cinema table and employees table using the intersection tables. Each branch has its own registered address. In the same way, we keep track of each employee's place of residence. Since we will use the same attributes to record addresses of cinema branches and addresses of employees, one table of addresses will suffice.

- id_address = unique identification of each address, used to link with cinema and employees tables
    - DT = serial: Serial is suitable type for a numeric unique identifier
- city = the name of the city where the branch is located or where the employee lives
    - DT = varchar(40): For entering a text string
- street = the street where the branch is located or where the employee lives
    - DT = varchar(45): For entering a text string
- house_number = house number where the branch is located or where the employee lives
    - DT = varchar(15): For entering a text string. We didn't use a numeric data type here because some addresses use a slash
- zip_code = For a better search of city addresses, but this attribute is optional. It can be easily found, for example, on the Internet
    - DT = varchar(6): For entering a text string. We didn't use a numeric data type here because the numbers in the zip-code are separated by a space

has_address:

The has_address intersection table is used to connect the cinema and address tables using the primary keys from each table.**1:1** relationship was used, when one cinema must have exactly one address and one address must be assigned to exactly one cinema (although someone could argue that one address could have multiple cinemas, in our opinion it is more than unlikely that one company would have multiple branches in one location).

- id_cinema = Foreign key referencing to the cinema table
    - DT = serial: Serial is suitable type for a numeric unique identifier
- id_address = Foreign key referencing to the address table
    - DT = serial: Serial is suitable type for a numeric unique identifier

opening_hours:

This table contains attributes: id, opening, closing. Contains information about the opening times of individual cinema branches. These attributes are linked to our cinema table using the intersection table.

- id_opening_hours = unique identification of each opening hours, used to link with the cinema table
    - DT = serial: Serial is suitable type for a numeric unique identifier
- opening = contains information about the opening hours of the given branch
    - DT = time: suitable for storing time data (the data type INT could be used here, but we think that the data type DATE format is more suitable)
- closing = contains information about the closing hours of the given branch
    - DT = time: suitable for storing time data

opens_closes:

The opens_closes intersection table is used to connect the cinema and opening_hours tables using the primary keys from each table.**1:N** relationship was used when one cinema must have one opening time, but one opening time can be assigned to n cinemas.

- id_cinema = Foreign key referencing to the cinema table
    - DT = serial: Serial is suitable type for a numeric unique identifier
- id_opening_hours = Foreign key referencing to the opening_hours table

- DT = serial: Serial is suitable type for a numeric unique identifier

films:

The film table records which films are currently broadcasted by individual branches. Attributes are stored in it: id, name of film, age limit, footage in minutes, premiere. These attributes are linked to our cinema table using the intersection table.

- id_film = unique identification of each film, used to link with cinema
  - DT = serial: Serial is suitable type for a numeric unique identifier
- films_names = the title of each movie
  - DT = varchar(80): For entering a text string
- age_limit = indicates from how many years the given film is accessible
  - DT = enum: data type with age selection(3,7,12,16,18)
- footage_minutes = length of the movie in minutes, so that everyone knows how much time they need to watch the movie
  - DT = smallint(4): integer type to write how many minutes it takes (there is no need to use INT, because the length of films should not be more than 3-4 hours – that means max. 3 digits + 1 spare)
- premiere = for some film titles, their first broadcast may also be indicated
  - DT = date: to insert the movie premiere date

has_film:

The has_film intersection table is used to connect the cinema and films tables using the primary keys from each table.**N:M** relationship was used, when one cinema can play n movies and one movie can be played by m cinemas.

- id_cinema = Foreign key referencing to the cinema table
  - DT = serial: Serial is suitable type for a numeric unique identifier
- id_film = Foreign key referencing to the films table
  - DT = serial: Serial is suitable type for a numeric unique identifier

genres:

The genres table is used to store the name of the genre and the ID of each genre, with the help of which we then assign the given genre to the given movie using the connection (intersection) table.

- dd_genre = unique identification of each position, used to link with films
    - DT = serial: Serial is suitable type for a numeric unique identifier
- genre = genre's name
    - DT = varchar(45): For entering a text string

has_genre:

The has_genre intersection table is used to connect the genres and films tables using the primary keys from each table.**1:N** relationship was used, when one movie must have exactly 1 genre and one genre can be assigned to n movies (the most difficult relationship to create, our database takes into account the fact that each movie is assigned one main genre that we use. The possibility of having multiple genres for one film seemed too complicated to us, and we think that it is not the main task of the project to make such a complex database).

- id_genre = Foreign key referencing to the genres table
    - DT = serial: Serial is suitable type for a numeric unique identifier
- id_film = Foreign key referencing to the films table
    - DT = serial: Serial is suitable type for a numeric unique identifier

employees:

The employees table is used to store records about employees, more precisely their ID, which represents the primary key for us here, then it is username and password, when these two attributes are used for subsequent work with the database, finally we store the name and surname of the employee. Using connection tables, we record the address of the employee, his position (together with his salary) and in which cinema he works. All this using the already mentioned ID.

- id_employee = unique identification of each employee, used to link with other tables
    - DT = serial: Serial is suitable type for a numeric unique identifier
- username = login for the employee to the database

    - DT = varchar(45): For entering a text string
- password = login for the employee to the database
    - DT = varchar(45): For entering a text string
- first_name = employee's name
    - DT = varchar(45): For entering a text string
- last_name = employee’s last name
    - DT = varchar(45): For entering a text string

positions:

The position table is used to store the positions, salaries and ID of each position, with the help of which we then assign the given position to the given employee using the connection (intersection) table.

- id_position = unique identification of each position, used to link with Employees
    - DT = serial: Serial is suitable type for a numeric unique identifier
- name = position’s name
    - DT = varchar(45): For entering a text string
- salary = position’s salary
    - DT = decimal(11,2): For entering the salary of each position (we use decimal to record whole crowns and pennies too)

has_position:

The has_position intersection table is used to connect the employees and positions tables using the primary keys from each table.**1:N** relationship was used, where one employee must have exactly one position and one position can be performed by n employees (in our list of job positions, an employee may perform only one position).

id_employee = Foreign key referencing to the employees table

    - DT = serial: Serial is suitable type for a numeric unique identifier
- id_position = Foreign key referencing to the positions table
    - DT = serial: Serial is suitable type for a numeric unique identifier

works:

The works intersection table is used to connect the employees and cinema tables using the primary keys from each table.**1:N** relationship was used, when exactly one employee has to work in one cinema and one cinema has n employees (although a person can have multiple jobs, our database only stores records for CinemaVillage, where an employee can only work at one location).

- id_cinema = Foreign key referencing to the Cinema table
    - DT = serial: Serial is suitable type for a numeric unique identifier
- id_employee = Foreign key referencing to the Employees table
    - DT = serial: Serial is suitable type for a numeric unique identifier

address:

As already mentioned, the address table is used to store the addresses of employees and cinemas. See above.

employee_has_address:

The employee_has_address intersection table is used to connect the employees and address tables using the primary keys from each table.**1:N** relationship was used, because one employee must have exactly one address and one address can be assigned to n employees (one employee has exactly one address of permanent residence, while several people can work from one block of flats).

- id_employee = Foreign key referencing to the employees table
    - DT = serial: Serial is suitable type for a numeric unique identifier
- id_address = Foreign key referencing to the address table
    - DT = serial: Serial is suitable type for a numeric unique identifier

Mandatory or optional relation

In order for our database to make sense, we have all the tables connected using a mandatory relational relationship. All tables are necessary and important.

Is database in 3rd normal form?

Table satisfies the third normal form if it satisfies the first and second normal forms.

All attributes in our table are atomic that is, all attributes are separate and carry one piece of information.

All our non-key attributes in individual tables are completely dependent on the primary key of that table.

We split our tables into other tables, linked them using foreign keys, due to this, we have removed the transitive dependency and satisfied the third normal form (we separated movies and movie genres, they created two tables that we linked using a foreign key).

Functional requirements

User identification for logging into the system

A notification is sent to an employee whenever his data are changed

If the employee displays cinema branch, system writes out additional information about the cinema.

Non-functional requirements

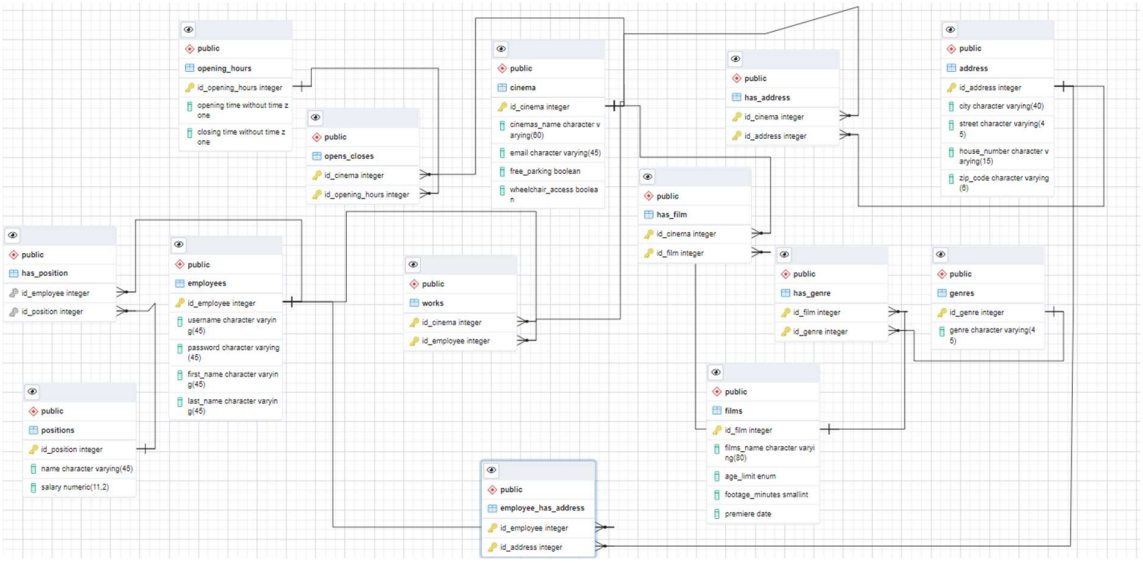
Employees are not allowed to update their personal information

Every unsuccessful attempt by user to insert incorrectly entered values will be denied

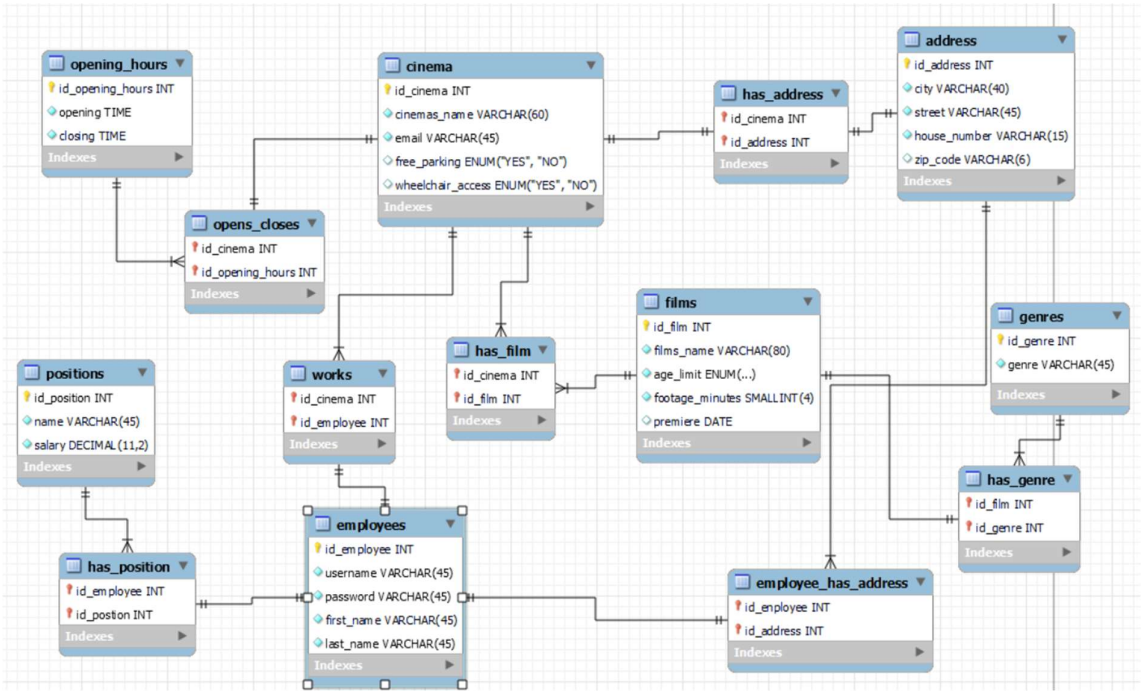
The system allows processing requests in real time

Database design and diagrams

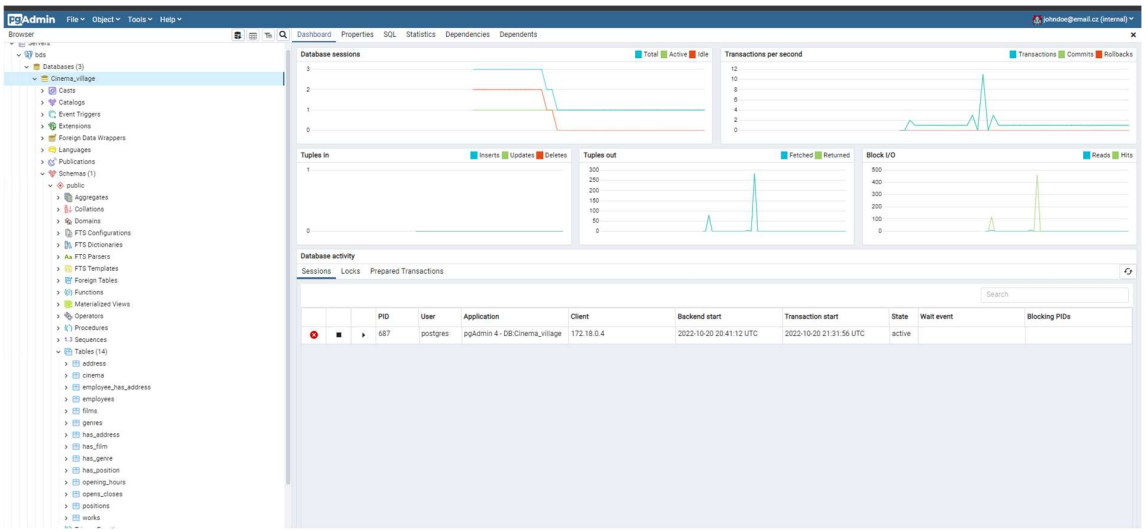
From PostgreSQL



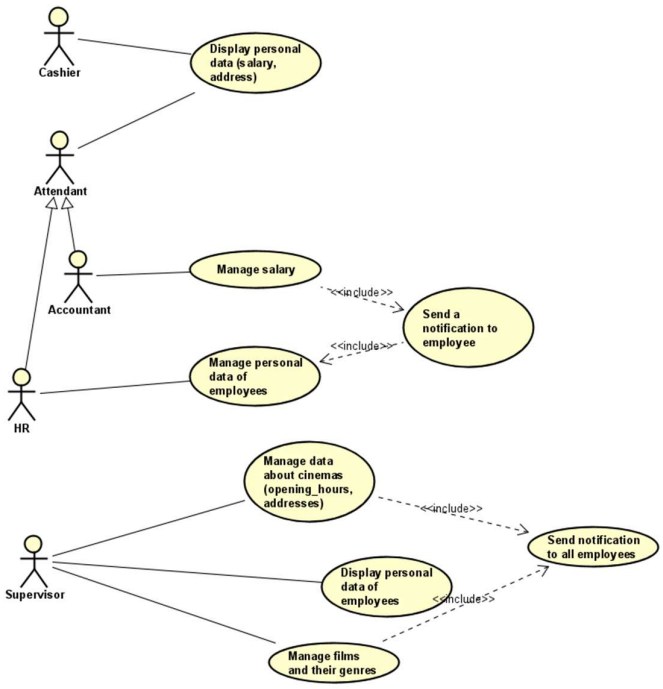
From MySQL Workbench



Screenshot from the pgAdmin



Use-case diagram



System context diagram